

Build **Swap** as a visual seat-exchange PWA

The strongest version of this product is **“see your seat, find a better fit, and arrange an approved swap”**—not another list of “upper berth available” posts.

Your differentiator should be an interactive coach or cabin map: users see exactly where they are, where they could move, and what changes—window access, group proximity, berth position, or distance from exits and toilets.

I can help implement this, but I can’t open or modify the linked GitHub repository from this chat. Upload the repository ZIP, excluding secrets and `.env`` files, so I can work against your actual codebase. I also haven’t inspected the competitor sites, so the proposal below is a product direction rather than a verified comparison of their features.

1. The core experience: map first, labels second

Find your journey

Support two entry points:

- **Train:** country → operator → train number → departure date → boarding and destination stations.
- **Flight:** airline → flight number → departure date → flight segment.

Then ask for the user’s coach/cabin and seat. Booking imports can come later; manual entry should always work.

See your actual position

For trains

- A whole-train strip showing coaches.
- Tap a coach to open its compartment layout.
- Show berth stacks visually, not just codes such as ``SU`` or ``LB``.
- Mark doors, toilets, aisles and windows where verified layout data supports them.
- Highlight nearby family/group members.
- Explain labels on tap: **“Upper berth, inside bay 4.”**

For flights

- A scrollable cabin map with actual row numbering and aisle configuration.
- Highlight windows, aisles, exit rows, galleys and lavatories where known.
- Show cabin boundaries and restricted seats.
- Never imply that a seat is available merely because nobody has registered it.

Show the proposed swap visually

``text

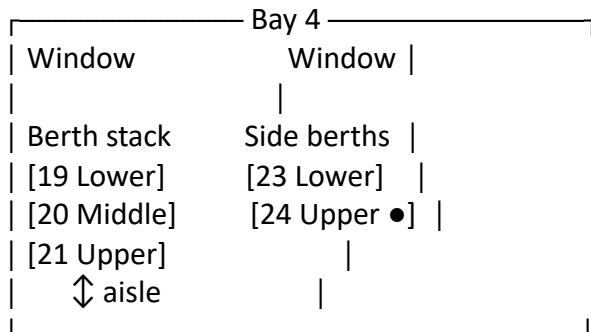
SWAP

Journey details

Mumbai → Delhi

Coach B3 · AC 3-tier

[B1] [B2] [B3 — You] [B4] [B5]



● Your seat ◇ Potential match

YOUR MOVE

24 Side Upper → 19 Lower

Same coach · Same bay · Overlapping journey

[Preview both seats] [Request swap]

...

Illustrative UI only; production maps must use the correct coach template.

The comparison should emphasize outcomes:

> **Move from an upper berth to a lower berth, staying in the same coach.**

Or:

> **Move next to your travel companion, three rows forward.**

2. Matching that understands the journey

A useful match is more than two compatible seat preferences.

Match requirements

- Same service instance and overlapping travel segment.
- Compatible coach/cabin and ticket class.
- Both users' seat preferences satisfied.
- No known accessibility, exit-row or operator restrictions violated.
- Both users agree explicitly.

Ranking

Rank eligible matches by:

1. Keeping families or groups together.
2. Meeting accessibility-related preferences without requiring public medical details.
3. Mutual seat preference improvement.
4. Shorter walking distance.
5. Journey-verification confidence.

Start with ****two-person swaps****. Add three-way swaps later; they introduce considerably more coordination and failure risk.

****Use an honest transaction flow:****

`Request → Accepted by both → Operator/crew confirmation, if required → Completed`

App acceptance must not be presented as an official seat reassignment.

3. Seat maps: build a data system, not a collection of pictures

****Train number alone does not reliably identify the exact coach layout. Airline alone does not identify the aircraft layout.**** Dates, routes, equipment substitutions and operator-specific configurations matter.

Use a resolution pipeline:

``text

Journey

- Operating service
- Scheduled equipment / coach composition
- Operator-specific layout variant
- Seat map
- User-confirmed seat

``

Data strategy

| Mode | Needed inputs | Practical approach |

|---|---|---|

| Trains | Operator, train number, date, segment, coach/class | Authorized service data plus curated coach-layout templates |

| Flights | Operating carrier,

| Mode | Needed inputs | Practical approach |

|---|---|---|

| Trains | Operator, train number, date, segment, coach/class | Authorized service data plus curated coach-layout templates |
| Flights | Operating carrier, flight number, date, aircraft and cabin configuration | Licensed flight/seat-map data where available, with operator-specific templates as fallback |

Make map accuracy visible

Every map should display one of these statuses:

- **Verified layout:** supplied by an authoritative source for this journey.
- **Expected layout:** matched to scheduled equipment; subject to change.
- **Illustrative layout:** a generic template that the passenger must confirm.

If the system cannot confidently resolve the layout, ask the user to select or confirm it.
Never invent seat positions or silently present a generic map as exact.

Also distinguish:

- **Your seat**
- **Passenger open to swapping**
- **Swap requested**
- **No information**

“No information” must not look like “empty seat.”

4. Monetization: charge for convenience, not ownership of seats

Passengers generally cannot assume they have permission to resell an assigned seat.
Operator rules differ, so **do not launch as an unrestricted seat marketplace.**

Recommended business model

Offering	Free	Paid
View supported seat maps	Yes	—
List seat and preferences	Yes	—
Request and accept a basic swap	Yes	—
Basic match notifications	Yes	—
Coordinate a larger travel group	Limited	Trip pass
Save household travel preferences	Limited	Subscription
Travel-agent group dashboard	—	Business subscription
Embedded seat coordination for operators	—	B2B licensing

Start with free matching and test a one-trip group-coordination pass. Subscription value is likely stronger for frequent travelers than occasional passengers.

Avoid paywalling the ability to respond to a request: a paying customer still needs the other person to participate.

Do not promise paid “priority” will produce a swap. Show clearly what is being purchased, and provide an appropriate refund policy.

Additional revenue later

- Disclosed travel-service affiliate links.
- Operator-approved seat-upgrade integrations.
- White-label group seating tools for tour companies.

Avoid intrusive ads inside the seat map or swap-confirmation flow.

5. Visual design direction

Mobile navigation

****Journey · Seat Map · Matches · Inbox · Profile****

The seat map should be the main workspace, not a secondary feature.

Essential screens

****1. Home****

- “Find a seat that fits your journey.”
- Train / Flight switch.
- Journey search.
- A small interactive map preview.
- Clear explanation: ****“Passenger coordination—not an official booking change.”****

****2. Journey workspace****

- Journey details and data freshness.
- Coach or cabin selector.
- Large, zoomable seat map.
- A bottom sheet showing selected-seat details.

****3. Swap preview****

- Side-by-side “Your current seat” and “Proposed seat.”
- Both locations highlighted on the map.
- Benefits and trade-offs.
- Any approval requirements.

****4. Matches****

Use visual cards:

- > **Sit together**
- > Two seats in the same row
- > Current: 18A + 24C → Proposed: 18A + 18B
- > **View on map**

5. Swap coordination

- Structured accept/decline actions.
- Optional in-app messaging.
- Report/block controls.
- Clear expiry and cancellation states.
- Completion confirmation from both travelers.

Accessibility requirements

- Seat status indicated with icons and text, not color alone.
- Keyboard and screen-reader navigation.
- Large touch targets.
- Reduced-motion support.
- A list alternative to the visual map.

For train berth stacks, use a **top-down compartment map plus a small elevation view**. This makes upper/middle/lower positions understandable without forcing users to interpret a pseudo-3D scene.

6. Suggested technical architecture

Keep your repository's existing framework if it is suitable. For a greenfield implementation:

Layer Recommendation
--- ---
Web/PWA Next.js, React, TypeScript
UI Tailwind CSS with accessible components
Seat rendering Interactive SVG
Database PostgreSQL
Authentication/storage/realtime Supabase or equivalent managed services
Matching Server-side rules and background jobs
Notifications In-app, email, supported web push
Payments Provider selected for launch markets
Monitoring Error tracking and privacy-conscious
Layer Recommendation
--- ---
Monitoring Error tracking and privacy-conscious product analytics
Data integrations Separate adapters for each train, flight and seat-map provider
Deployment Managed web hosting plus a background worker for matching and notifications

****Keep the seat-map engine independent of transport providers.**** This lets you add countries and operators without rebuilding the interface.

7. Core data model

The most important distinction is between a ****journey****, a ****physical seat****, and a ****passenger's use of that seat over a segment****.

On a train, the same berth may belong to different passengers on different sections of the route.

Entity Purpose
--- ---
`operators` Airlines and railway operators
`service_instances` A particular train or flight departure
`service_stops` Ordered stops with local time zones
`layout_templates` Versioned aircraft/coach configurations
`equipment_assignments` Expected or verified layouts for a service
`seats` Seat identifiers, positions and features
`traveler_journeys` User, service, boarding stop and destination
`seat_claims` User's reported seat and verification status
`swap_preferences` Desired features and acceptable trade-offs
`swap_requests` Participants, proposed changes and expiry
`swap_events` Audit history of acceptance and cancellation
`reports` Abuse and inaccurate-seat reports

Seat-map format

Use one renderer with transport-specific geometry:

```
```ts
type SeatFeature =
 | "window"
 | "aisle"
 | "middle"
 | "lower_berth"
 | "middle_berth"
 | "upper_berth"
 | "side_berth"
 | "exit_row";

interface Seat {
 id: string;
 label: string;
}
```

```

 compartmentId?: string;
 position: {
 x: number;
 y: number;
 level?: number;
 };
 features: SeatFeature[];
 restrictions: string[];
 }

interface SeatMap {
 id: string;
 version: number;
 transport: "train" | "flight";
 operatorId: string;
 equipmentCode: string;
 width: number;
 height: number;
 seats: Seat[];
 fixtures: Array<{
 type: "door" | "toilet" | "galley" | "aisle" | "stairs";
 x: number;
 y: number;
 width: number;
 height: number;
 }>;
}
...

```

Keep **layout geometry** separate from **live passenger participation**. A cached map should never imply that a cached swap offer is still available.

### ### Segment-aware matching

For ordered stop indexes, two travel intervals overlap when:

```

```ts
const overlaps =
  Math.max(a.boardingIndex, b.boardingIndex) <
  Math.min(a.destinationIndex, b.destinationIndex);
...

```

That is only a preliminary test.

For the first release, prefer **identical boarding and destination segments**. Partial-overlap swaps require explicit start/end boundaries, reliable seat ownership throughout the exchange, and operator-compatible rules. Introduce those later.

8. Trust, safety and privacy are product features

Verification

Use progressively stronger verification:

1. ****Unverified:**** seat entered manually.
2. ****Journey evidence checked:**** supported booking evidence has been checked.
3. ****Operator verified:**** confirmed through an authorized integration.

Do not label someone “verified” merely because they uploaded an image.

If booking uploads are introduced:

- Extract only necessary information.
- Avoid retaining complete tickets unnecessarily.
- Never expose booking references, ticket barcodes or legal names to other travelers.
- Apply retention limits and deletion controls.

Public visibility

Before mutual acceptance, show:

- Anonymous traveler alias.
- Relevant seat and journey segment.
- Swap preferences.
- Verification level.

Do not show a publicly searchable passenger manifest. Limit journey participation visibility to the relevant service, with abuse controls.

Swap rules

- Users can withdraw before completion.
- Requests expire before they become stale.
- Accepted seats cannot be committed to multiple active swaps.
- Equipment changes trigger revalidation.
- Exit-row and accessibility restrictions cannot be bypassed through the app.
- The app never changes a booking unless an authorized integration actually supports that operation.

Use database transactions and uniqueness constraints for commitments—not just disabled buttons.

****For flights especially, display the crew/airline approval requirement clearly.**** Passenger agreement alone may not authorize moving.

9. PWA behavior that genuinely helps travelers

Works offline

- Open the app
- View a previously loaded journey and seat map.
- Read saved swap instructions.
- See the last known request status, clearly marked with its update time.
- Prepare preferences locally.

Requires a connection

- Publish a seat.
- Send or accept a swap request.
- Confirm availability.
- Make a payment.
- Complete a swap.

****Never show an offline acceptance as confirmed.**** Reconnect, revalidate the seats, and obtain server confirmation first.

PWA essentials

- Installable manifest and app icons.
- Fast-loading application shell.
- Offline fallback screen.
- Opt-in push notifications where supported.
- Email/in-app fallbacks for unsupported notifications.
- Accessible touch controls and safe-area support.
- Minimal caching of sensitive journey information, cleared on logout.

10. Launch globally by design—not everywhere at once

Seat swapping needs ****multiple participating passengers on the same departure****, not just a large total user count.

Start with a concentrated market, then expand.

Recommended rollout

Stage Scope Success condition
--- --- ---

Pilot	Selected Indian train services and validated coach types	Passengers can find and complete useful swaps
Expansion	More supported train layouts and routes	Reliable maps and repeatable user acquisition
Flight beta	Selected airlines/configurations with reviewed rules	Accurate layouts and clear approval workflows
International	Additional countries through provider adapters	Local data, payments, language and policy support

Build international foundations immediately:

- Translated interface strings.
- Local time zones and date formats.
- Operator-scoped service identifiers.
- Right-to-left layout support.
- Configurable currencies and regional payment availability.
- Country/operator-specific swap rules.

But label unsupported journeys honestly. ****“Global”** should describe the architecture and ambition—not an unsupported coverage claim.**

Acquisition that creates matching density

- Shareable journey invitation links.
- “Invite your travel group” onboarding.
- Partnerships with tour organizers and travel communities.
- Useful coach-map pages that lead into journey-specific matching.
- Operator-approved distribution opportunities.

Shared links must not expose booking references, passenger names or private seat details.

11. What the first release should contain

Ship first

- Train/flight journey selection.
- A small library of validated layout templates.
- Visual seat selection.
- Seat claim and preference setup.
- Two-person matching.
- Request, accept, cancel and expire flows.
- Structured coordination messages.
- Reporting and blocking.
- Installable PWA with cached maps.
- Admin tools for layout corrections and moderation.
- Analytics for the full matching funnel.

Defer

- Three-way swaps.
- Passenger-to-passenger seat payments.
- Claims of universal live seat-map coverage.
- Automated booking changes without operator integrations.
- Complex ticket-document processing.
- Paid subscriptions before matching demonstrates value.

A visually impressive map is important, but ****correct seat positions and trustworthy swap states are the release gate.****

12. Implementation structure

Adapt this to the existing repository rather than replacing its stack unnecessarily:

```
```text
src/
├── app/
│ ├── journeys/
│ ├── matches/
│ ├── inbox/
│ └── account/
├── components/
│ ├── seat-map/
│ │ ├── SeatMap.tsx
│ │ ├── CoachStrip.tsx
│ │ ├── BerthElevation.tsx
│ │ ├── SeatLegend.tsx
│ │ └── SwapPreview.tsx
│ └── journey/
├── lib/
│ ├── matching/
│ ├── swap-state/
│ ├── providers/
│ │ ├── trains/
│ │ └── flights/
│ └── validation/
├── data/
│ └── layouts/
└── workers/
 ├── match-notifications.ts
 └── expire-requests.ts
```
```

Critical tests

- Seat labels and geometry match each validated template.
- Different departure dates never cross-match.
- Codeshare flight numbers resolve to the correct operating service.
- Equipment changes invalidate affected proposals.
- Concurrent acceptance cannot double-commit a seat.
- Expired requests cannot be accepted.
- Offline screens cannot present stale availability as current.
- Users cannot access another traveler's private booking information.

13. Measure usefulness before monetization

Track:

1. **Seat-map resolution rate:** how often the correct layout can be identified.
2. **Journey activation:** users who add a seat and preferences.
3. **Match coverage:** percentage of active listings with at least one eligible counterpart.
4. **Request acceptance:** how often a proposed exchange works for both passengers.
5. **Confirmed completion:** swaps both travelers report as completed.
6. **Time to useful match:** measured against time remaining before departure.
7. **Map correction rate:** how frequently users report inaccurate layouts.
8. **Revenue and support cost per journey:** whether monetization is sustainable.

Separate **"no compatible passenger joined"** from **"matching failed."** The first is a distribution problem; the second may be a product or data problem.

14. Build order: prove the visual experience first

Milestone 1 — Interactive product demo

Build a complete, clearly labeled demo using curated sample journeys:

- Train coach strip and compartment map.
- Flight cabin map.
- Select your seat.
- Set preferences visually.
- Preview a proposed exchange.
- Responsive mobile layout and installable PWA shell.

Acceptance criterion: a user can understand both seats' locations without relying on berth codes or seat descriptions.

Milestone 2 — Real passenger coordination

Add:

- Authentication and database.
- Journey-specific seat claims.
- Server-side matching.
- Atomic request acceptance.
- Expiry, cancellation and notifications.
- Reporting, blocking and moderation.

****Acceptance criterion:**** two users can coordinate a swap without exposing booking credentials or creating conflicting commitments.

Milestone 3 — Reliable journey data

Add provider adapters, layout resolution and an administrative review workflow.

Every resolved map should carry:

- Source.
- Layout version.
- Last checked time.
- Confidence status.
- Applicable service/equipment identifiers.

****Acceptance criterion:**** unsupported equipment produces a clear fallback—not a fabricated “exact” map.

Milestone 4 — Monetization

Introduce a group trip pass only after basic matching proves useful. Show prices and benefits explicitly; do not sell guaranteed outcomes.

15. Positioning for the finished product

Homepage headline

> ****A better seat starts with seeing your options.****

Supporting copy

> Explore your coach or cabin, find travelers with compatible preferences, and coordinate a seat swap—subject to operator rules.

Primary actions

****Find my journey** . **Explore a sample map****

Your defensible advantage

Not simply “seat swapping,” but:

- ****Understandable visual layouts****
- ****Accurate journey-specific matching****
- ****Group seating coordination****

- ****Transparent verification and approval****
- ****Reliable coverage that expands operator by operator****

Avoid claiming “the world’s largest” or “guaranteed swaps” without evidence.

Next step: turn this into code

****Upload the `ayodhyamohanthyswap` repository ZIP, excluding secrets, `.env` files, dependencies and build output.**** If uploading the whole repository is difficult, start with:

```
```text
package.json
README.md
src/ or app/
public/
database schema or migrations
framework configuration
.env.example containing placeholder values only
```
```

With the source available, I can produce file-by-file changes for the first working milestone:
****the visual train/flight seat-map experience, swap preview, and PWA foundation****,
preserving the existing stack where practical.

****Current status:**** this is a proposed product and implementation plan; no repository changes or live provider integrations have been made.